



Rapport projet d'Algorithmie des graphes

Résolution du problème du voyageur de commerce

Tristan GROULT, Louis GUERAIN, Kibandou BIGNANG

2 janvier 2026

Sommaire

1 Notice d'utilisation **3**

2 Algorithme **3**

2.1 Point le Plus Proche (PPP) 3

2.2 Point le Plus Proche Optimisé (OptPPP) 5

2.3 Arbre couvrant de poids minimum (OptPrim) 5

2.4 Évaluation et séparation progressive ("branch and bound") (HDS) 5

1 Notice d'utilisation

Ce projet dépend des bibliothèques `numpy` et d'autres modules standards. Il dépend aussi de `flask` pour l'application web.

```
pip install numpy # (tout)
pip install flask # (web app)
```

Listing 1 – Installation des dépendances

Ou plus simplement :

```
pip install requirement.txt
```

2 Algorithme

2.1 Point le Plus Proche (PPP)

Algorithme : Procédure PPP (Point le Plus Proche)
Description : Construit un cycle hamiltonien en insérant itérativement le point le plus proche du cycle actuel.
Entrées :
- L : Liste de points $P_1, \ldots, P_n$
- Q : Un point de départ choisi dans L
- D : La matrice des distances $n \times n$

Sorties :
- c : Un cycle hamiltonien
- l : La longueur du cycle

Début :

1. $c \leftarrow$ cycle trivial composé du seul point Q
2. $l \leftarrow 0$
3. Retirer Q de L
4. Tant que L n'est pas vide faire :
5. $\quad (Q_i, Q_j) \leftarrow \texttt{TrouverCoupleMin}(L, c, D)$
6. $\quad (c, l) \leftarrow \texttt{InsérerDansCycle}(c, Q_i, Q_j, D, l)$
7. $\quad$ Retirer Q_i de L
8. Retourner c, l

Algorithme : InsérerDansCycle
Description : Insère Q_i dans c soit entre Q_j et son successeur, soit entre son prédécesseur et Q_j, selon le coût minimal, et met à jour la longueur du cycle.
Entrées :

- c : Cycle partiel
- Q_i : Point à insérer
- Q_j : Point d'ancrage dans c
- D : Matrice des distances
- l : Longueur actuelle du cycle

Sorties : c (cycle mis à jour), l (nouvelle longueur)
Début :

1. Soit Q_{next} le successeur de Q_j dans c
2. Soit Q_{prev} le prédécesseur de Q_j dans c
3. $\text{coût}_{\text{droite}} \leftarrow D[Q_j, Q_i] + D[Q_i, Q_{next}] - D[Q_j, Q_{next}]$
4. $\text{coût}_{\text{gauche}} \leftarrow D[Q_{prev}, Q_i] + D[Q_i, Q_j] - D[Q_{prev}, Q_j]$
5. Si $\text{coût}_{\text{droite}} \leq \text{coût}_{\text{gauche}}$ alors :
6. Insérer Q_i entre Q_j et Q_{next} dans c
7. $l \leftarrow l + \text{coût}_{\text{droite}}$
8. Sinon :
9. Insérer Q_i entre Q_{prev} et Q_j dans c
10. $l \leftarrow l + \text{coût}_{\text{gauche}}$
11. Retourner c, l

Algorithme : TrouverCoupleMin
Description : Trouve le couple (Q_i, Q_j) tel que $Q_i \in L$, $Q_j \in c$ et la distance $D[Q_i, Q_j]$ est minimale.
Entrées :
— L : Liste des points non encore visités
— c : Le cycle partiel actuel
— D : La matrice des distances $n \times n$
Sorties : Q_i (point sélectionné dans L), Q_j (point d'ancrage dans c)
Début :

1. $d_{min} \leftarrow \infty$
2. $Q_i \leftarrow$ vide
3. $Q_j \leftarrow$ vide
4. Pour chaque point u appartenant à L faire :
5. Pour chaque point v appartenant à c faire :
6. Si $D[u, v] < d_{min}$ alors :
7. $d_{min} \leftarrow D[u, v]$
8. $Q_i \leftarrow u$
9. $Q_j \leftarrow v$
10. Retourner Q_i, Q_j

2.2 Point le Plus Proche Optimisé (OptPPP)

Section à compléter.

2.3 Arbre couvrant de poids minimum (OptPrim)

Section à compléter.

2.4 Évaluation et séparation progressive ("branch and bound") (HDS)

Section à compléter.